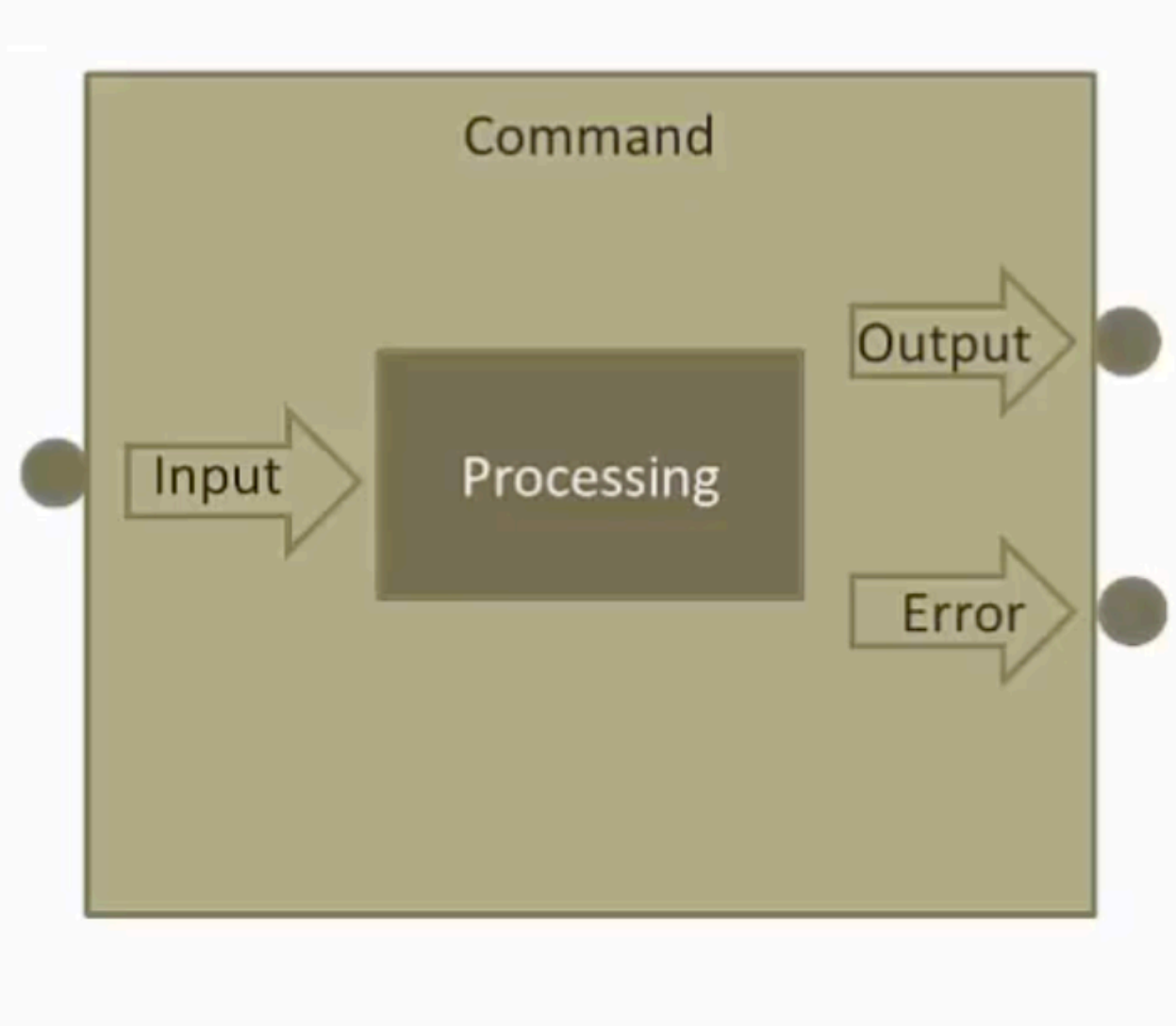
Composite Commands

Linux Command Philosophy

- Linux Commands follows the following philosophy,
  - Create **small**, **portable**, specialized programs that perform **one task** well.
  - Make this program capable of receiving input from, and redirect output to other programs.
- This way, Linux commands are like **LEGO** blocks.
- This philosophy has resulted in creation of a lot of small but very powerful when **joined together**.

Command Model

Input → Processing → Output/Error



Example:

```
mkdir dir/
```

Each one of them has its own stream

Function	Stream name	Stream Descriptor	Default Device
Input	stdin	0	keyboard
Output	stdout	1	screen
Error	stderr	2	screen

We can build composite commands through the following:

- **Sequential** Commands
- **Conditional** Commands
- **Command** Loops
- **Input/Output Redirection**
- **Pipes**
- **Command Argument Expansion**
- **Command Argument Quoting**

1.Sequential Commands

Independent Input/Output/Error

We can have multiple commands in the same line as follows:

```
$ <first command>; <second command> ; <third command>
$ echo "One"; sleep 10; echo "Two"
# lw ktbthom monfslen msh hyb2o 10 seconds bzbbt 7sb sor3ty f
lktaba
```

Using a sequential commands is useful when the first command takes long time to execute and we don’t want to wait until it is complete

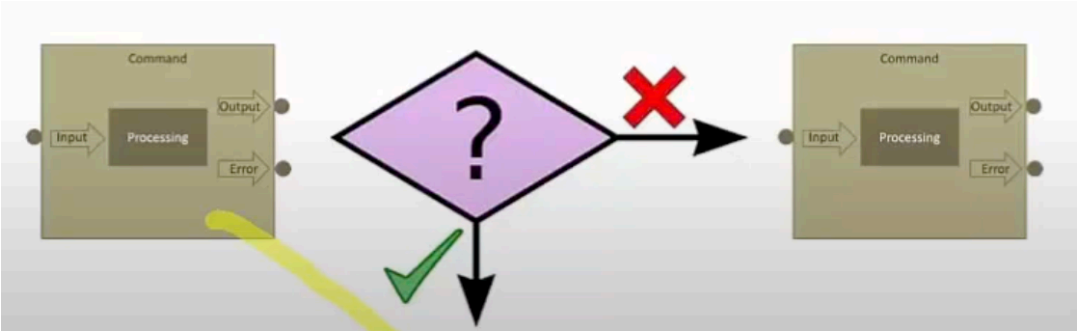
```
$make ; sudo make install
# m7taga root access 3shan l install bywz3 el files f amakn t
7t el root
# bdl mahstna el make w a3od abos kol shoywa
# lw command byakhod w2t tweel zy el make msln ! [Compilatio
n]
```

Note:

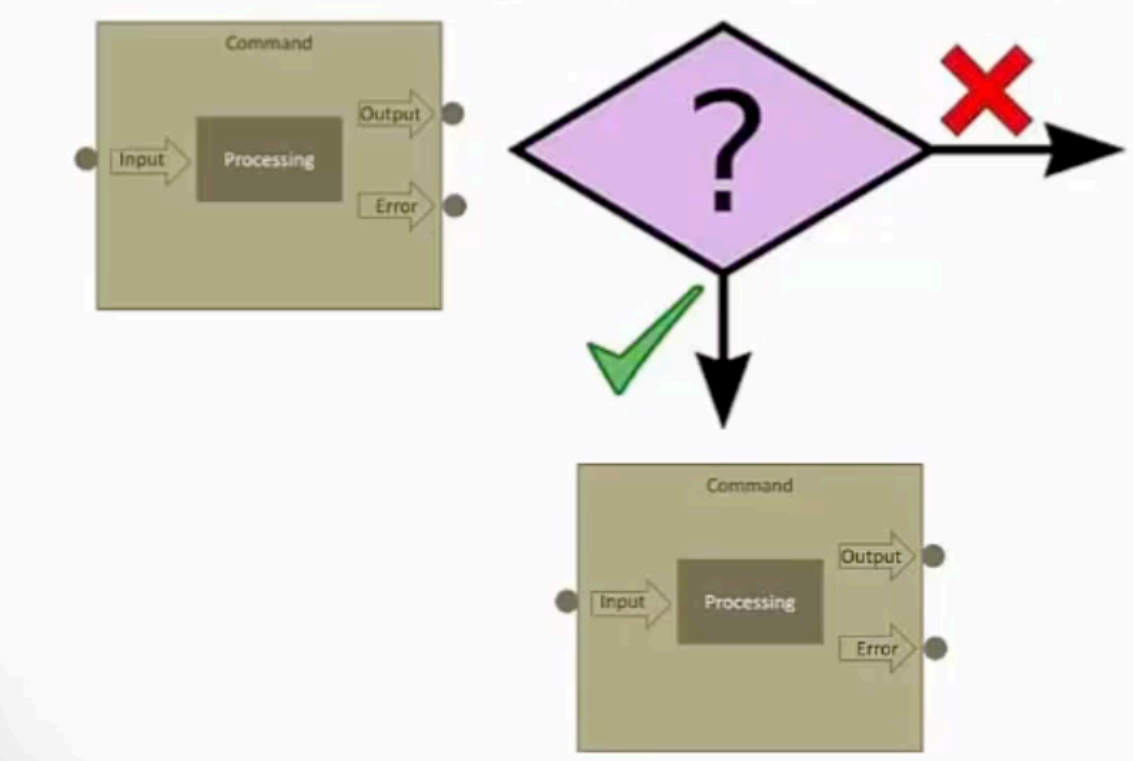
> The sequential commands just run after each other, they have independent Input/Output.

2.Conditional Commands

- **Oring** (″||″)
  - Second command will **only execute if the First Returns Failure.**
  - `$ cat <filename> || echo “File Not Found”.`
  - mmkn yb2a brdo eno ana b7awl a3ml copy l file wlw fshl yms7 el directory lly 3mltha
  - Used for Error Handling.
    - Input ⟶ Processing ⟶ Output ⟶ Conditional



- **ANDing** (″&&″)
  - Second command will **only execute if the First Returns successfully**
    - `$mkdir dir1 && cd dir1`
  - Continue as long as you are successful



3.Command Loops

We can build a loop when we want the command to execute multiple times (maybe on different files or so)

```
Examples:

$for file in *.txt
    >do
    >mv -v $file $file.old
    >done

    # 3adtn bn3ml elklam da f script bs it's good to know
```

We will discuss them in more detail in our Bash scripting lecture

4.Input/ Output Redirection

Standard Output Redirection

I can redirect Input/output/ Error to file [swa2 nfs el file aw diff files]

```
$ <command> > file (redirect output to file; overwrite)
$ date > file.txt

$ <command> >> file (redirect output to file; append)
$ date >> file.txt

$echo "Hello World" (Output goes to screen)
$echo "Hello World" > greeting.txt (Output goes to the file,
overwrite)
$ls -al /usr/bin >> file-listing.txt (Output goes to the fil
e, append)
$cat file1 file2 > combined-file.txt [Concatenate]
```

Note:

- Error Messages still go to the screen
- “>” Stands for (and can be replaced by) “1>” which means redirect output stream

Standard Error Redirection

```
$ <Command> 2> file (Redirect Error to file; Overwrite)
$ <Command> 2>> file (Redirect Error to file; Append)

$make (Both Output and Error messages go to screen)
$make 2> log (Output goes to Error messages go to file, overw
rite)
$make 2>> log (Output goes to stdout; Error messages go to fi
le; append)
```

Both Output & Error Redirection

```
Output and Error go to different files,
$ <command> > file1 2>file2 (output to file1, error to file2)
$ <command> >>file1 2>>file2 (output to file1, error to file
2)
```

```
$ <command> >file 2>&1 [stream num2 wadeh l stream num1]
$ <command> >>file 2>>&1

# short version of the same command
$ <command> &>file (both to file)  #kol l streams tro7 l file
$ <command> &>>file (both to file) #kol l streams tro7 l file
```

Standard Input Redirection

```
$ command < file (input to the command is read from the file)

Examples:
$ wc -l < log-file.log  # word count [char/word/line]

# 3ayz a3rf el code/log bta3y kam line msln

$ sort < log-file > sorted-log-file [alphabetically]
# mmkn agm3 kza log file f nfs el file msln !

$ mail ceo@company.com < resume
# h3ml script lw la2a severe problem hyb3tlk mail

$ spell < report.txt > error.log
# spelling errors
# mmkn a3ml report by3ml elkalam da
```

I/O Redirection With Devices

Common Devices for redirection

```
/dev/stdout:
/dev/stdin:
/dev/stderr:

/dev/null: [sink/Bala3a]
The device is useful for being a data sink. This is useful wh
en we want to discard the command
output (such as compiler long output)
msh 3ayz aktbo f file 3shan hyakhod storage

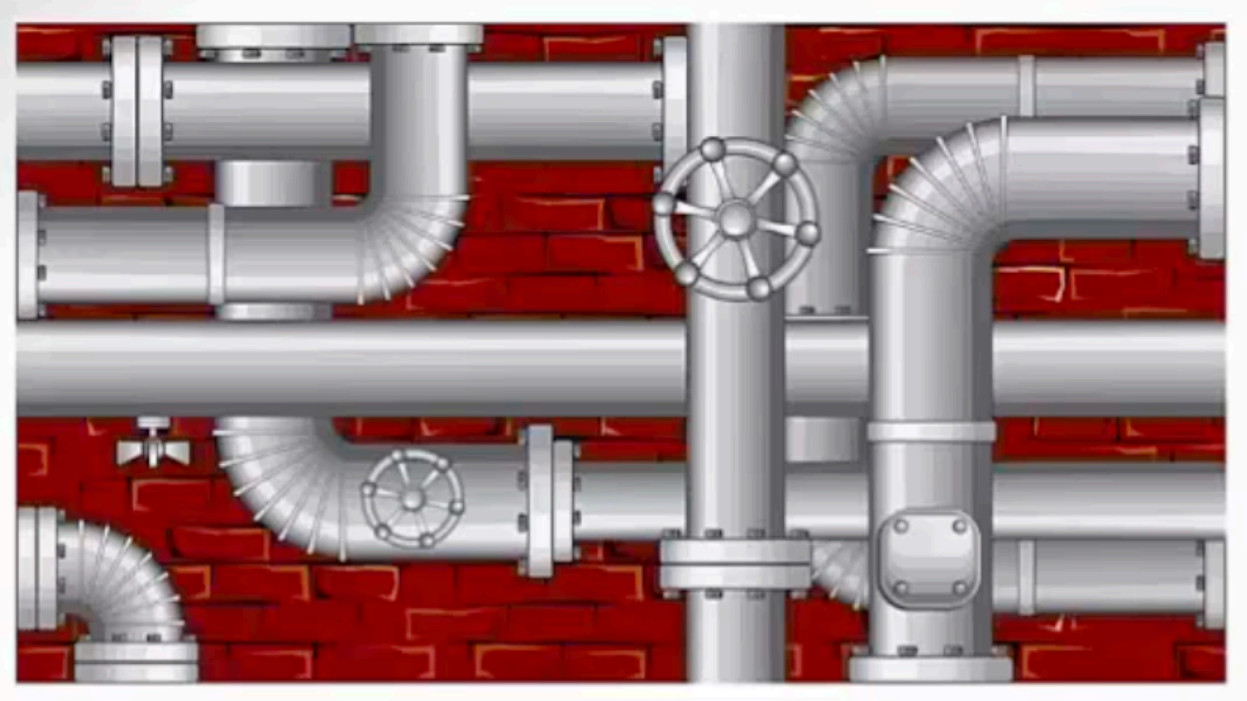
/dev/zero # fe /dev/one?
This device is useful as an input device to generate and infi
nite stream of zeros.

/dev/random/
This device is useful as an input device to generate random b
ytes. It may block.

/dev/urandom
This device is useful as an input device to generate quasi-ra
ndom bytes. It is non blocking

/dev/full
This device is used as an output device to simulate a full fi
le. It is used for testing
purposes.
```

Using Pipes

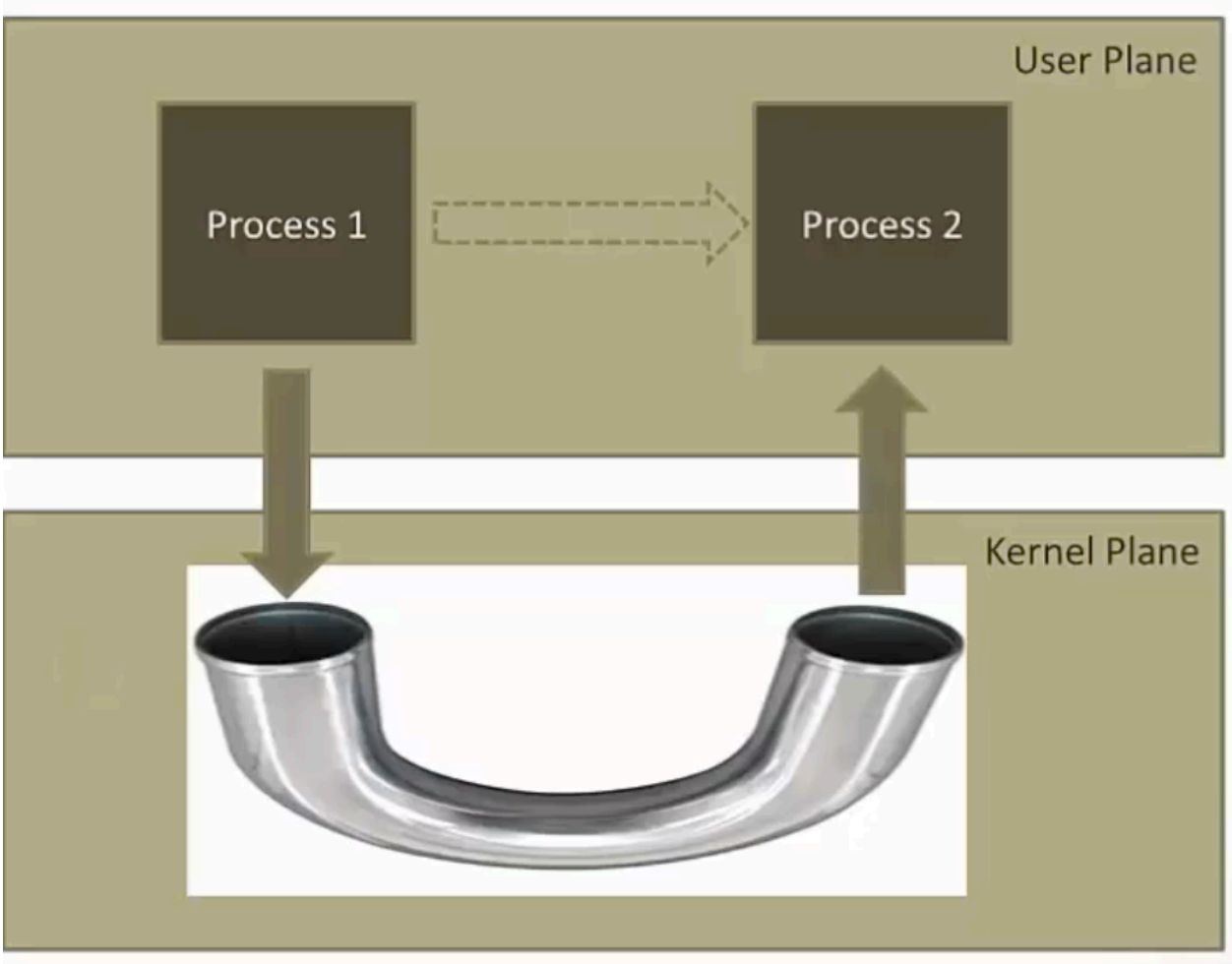


Process 1 want to give an input to process 2 ! How could we achieve that ! Through a pipe created in the Kernel Plane.

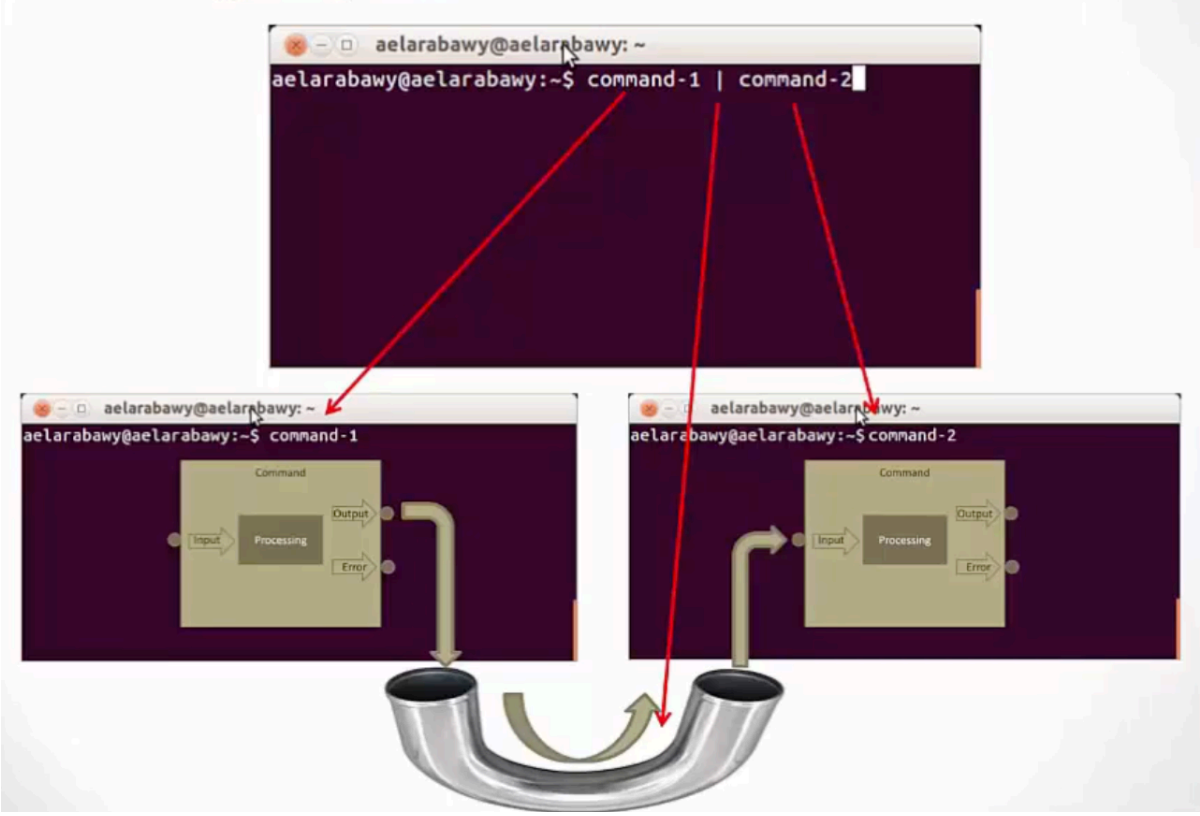
btkhosh f pipe/masora ttl3 k input.

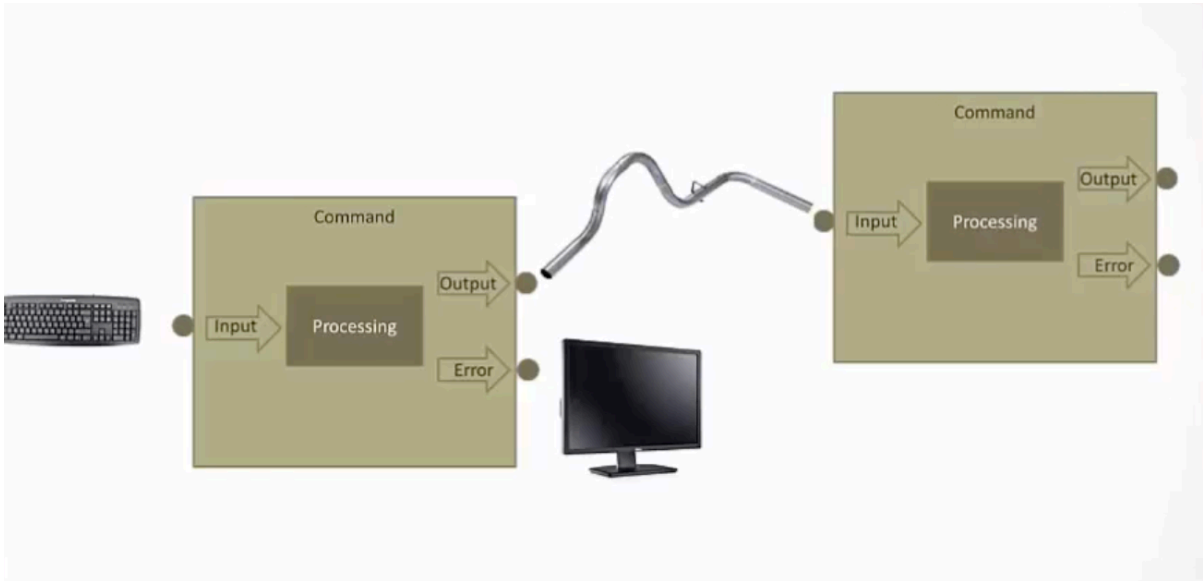
Pipe is a mechanism for communication between different processes.

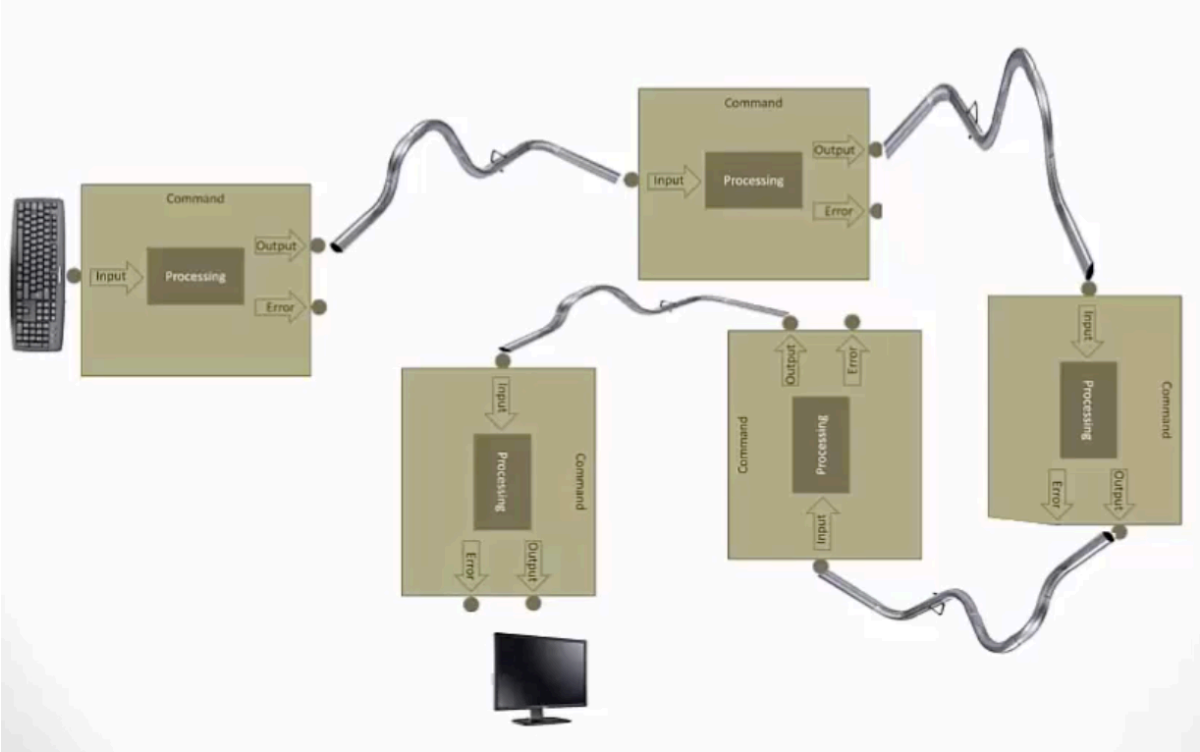
What is a pipe?



- A pipe is a mechanism in Linux Kernel that is used to enable one process to send info to the other process.
- This is called **IPC** (Inter Process Communication).
- A pipe is a **unidirectional** mechanism, so if you need data to flow in both directions, you will need two pipes (One for each direction).


aelarabawy@aelarabawy: ~
aelarabawy@aelarabawy:~$ command-1 | command-2
aelarabawy@aelarabawy: ~
aelarabawy@aelarabawy:~$ command-1
Command
Input
Processing
Output
Error
aelarabawy@aelarabawy: ~
aelarabawy@aelarabawy:~$ command-2
Command
Input
Processing
Output
Error


Command
Input
Processing
Output
Error
Command
Input
Processing
Output
Error




```
$ command-1 | command-2 [hyft7 internally two children shells
w y3ml pipe]
```

el Output stream bta3 process 1 hytn2l 3la pipe ll input bta3 Process 2.

- Pipes in Linux are a method for Communication between different processes (IPC)
- On the CLI, we can run two commands and connect them with a pipe, so output of one command feeds in the input of another one.
- Note that every command runs in a sub-shell of the shell issuing it (Child shell).

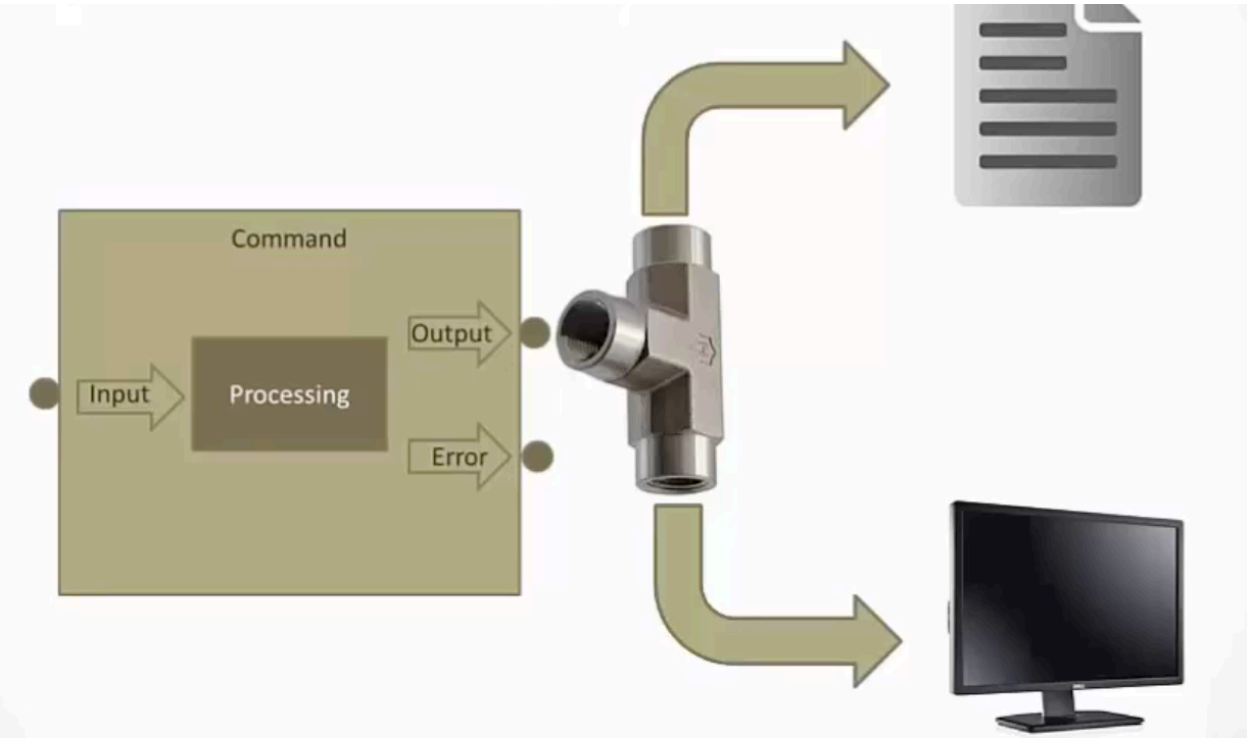
Using Pipes

```
$ <command1> | <command2> | <command3>

Exampls:
$ cat log-file.log | grep "Error"
```

```
$ man gzip | grep -i "compress" #case insensitive
$ cat log-file-1.log log-file-2.log | grep -i "error"
$ cat name-list.txt | sort
$ cat * | grep -i "error" | grep -v "severe" | sort > file.lo
g
$ cat resumes | mail bill-gates@microsoft.com
```

More With Pipes (tee (T) Command)



tee command



```
$ <Command> | tee <list of Sinks>
```

The tee command sends the output to a set of files as well as the stdout

Examples:

```
$ make | tee make.out.txt (output to the screen and the file)
$ make | tee -a make.out.txt (same, but uses append mode)

$ date | tee file1 file2 file3 file4 (date is sent to the 4 f
iles & screen)
$ make | tee make.out.txt >> file2 (output goes to make/out.t
xt and appends file2)
$ make | tee make.out | grep "error"
```

More With Pipes (yes command)

```
$ yes <string> | <command>
```

The command “yes” passes a string to the command prompt

The default string is “y”.

Examples:

```
$ yes | rm -r ~/project-folder/
$ yes "" | make config (pass empty string to accept defaults)
```

6. Commands Arguments Expansion Wild Card Expansion (*, ?, ...)

- Discussed in a previous Lecture.
- Applies for File and Directory names.

```
Examples:
$ cp *../project/
$ cat * > log-files
$ echo * > $ ls
```

7. Commands Arguments Expansion Tilde Expansion (~)

The Tilde is expanded to the path of the user home folder /home/<username>

Examples:

$ echo ~ → home/ramy

Commands Arguments Expansion Parameter Expansion ($ and ${})

The **parameter expansion** is used to evaluate shell or environment variables

Examples:

$ echo $PATH

$ echo ${PATH}

$ MY_NAME=TOM

$ echo “My Name Is $MY_NAME”

Commands Arguments Expansion Arithmetic Expansion ($(()))

The Arithmetic Expansion is used to put arithmetic expressions to be evaluated

Examples:

$ echo $((5 + 6)) → 11

$ echo $((((5**2)) * 3)) → 75

$ echo $((5/2)) → 2

Commands Arguments Expansion Brace Expansion ({})

The Brace Expansion results in a set of values as described inside the braces

Examples:

```
$ echo abc-{A,B,C}def -> abc-Adef abc-Bdef abc-Cdef
$ echo a{A{1,2}, B{3,4}}b -> aA1b aA2b aB3b aB4b
$ mkdir a{A,B} -> aA, aB
```

Similarly, you can use the expansions

{1..5}

{A...Z}

{Z...A}

8. Command Argument Quoting (“”)

Double quotes,

- Protect strings (spaces, wild cards, tilde, ...)
  - $ echo “*” ⟶ *
- Protect from command substitution.
  - $ echo “ls” ⟶ ls
- Protect from Arithmetic Expansion.

    - $ echo “$((5+6))” → $((5+6))
- Does no stop variable retrieval
    - $ echo “My Name Is $MY_NAME” → My Name Is Tom

Examples:

- $ echo “cp *.*” → cp “.”
- $ echo “$HOME” → /home/ramy

Command Argument Quoting Single Quotes (‘ ’)

Single Quotes,

- Similar to double quotes except that it also protects variable retrieval.

Examples:

$ echo ‘$PATH’ → $PATH

$ echo ‘My Name Is $MY_NAME’ → My Name Is $MY_NAME

Command Argument Quoting Back Quotes (‘’)

Back Quotes are used to do commands inside commands/strings

Examples:

- `$ cd /lib/modules/`uname -r``
- is equivalent to
- `$ cd /lib/modules/3.11.0.15-generic`

Can also be replaced by $(<command>) so,

`command` is the same as $(command)

Example:

```
$ cd /lib/modles/$(uname -r)
```

Other Examples:

```
$pushd `pwd`
```

```
$ touch `date+%m-%d-%Y`
```